🚀 Ultimate Prompt Engineering Guide

A practical guide to designing prompts that are clearer, more controllable, and more useful, with a real-world focus on learning, software development, and automation.

Author **MC. David Alejandro Garcia Garcia**

Clarity · Context · Format · Iteration · Validation · Reliability

1) What is Prompt Engineering?

Prompt Engineering is the practice of designing instructions so that an AI model produces more useful, relevant, and consistent responses.

> A good prompt reduces ambiguity, defines the goal, provides enough context, specifies the expected output format, and makes clear what to do if information is missing.

> Think of a prompt as a short specification: it is not just "asking something", it is designing an input that correctly guides the model's behavior.

2) Anatomy of a High-Quality Prompt

🎯 Instruction

What you want the model to do.

```
Summarize the text in 5 bullet points.
```

🌍 Context

Information that helps the model respond better.

```
This is for an executive meeting and the audience is non-technical.
```

📥 Input Data

The text, code, or question the model will work on.

```
### Text ###
[content here]
###
```

📤 Output Format

How the response should be structured.

```
Return JSON with: title, summary, risks
```

🧪 Examples

Demonstrations of expected inputs and outputs.

```
Input: "Hello"
Output:
{"language":"en","tone":"neutral"}
```

✅ Validation / Fallback

What to do if information is missing or confidence is low.

```
If data is missing, state your assumptions.
If you cannot verify it, say so explicitly.
```

> Recommended formula: **Role + Objective + Context + Input + Constraints + Format + Validation**

3) Prompt Categories

🌐 Open-ended

When: Ideation, exploration.

```
What frontend frameworks should I learn?
```

✅ Close-ended

When: Precise answers.

```
What is the primary database in AWS?
```

🧩 Multi-part

When: Multiple outputs in one prompt.

```
Suggest some technologies I can use for data visualization
```

That's the first part, now the second:

```
And list their strengths and weaknesses
```

🎯 Scenario-based

When: The scenario is used to set the context within which we want the information that we are querying for. Scenario-based prompts are useful for testing problem-solving skills or exploring hypothetical situations.

```
I'm hosting a birthday party. They are all between 11 and 13 years old.
```

That's the scenario. The second line is the question:

```
Based on this scenario, what do you think I should organize? What food do you suggest? I have to ensure that they all have a good time.
```

💬 Opinion-based

When: Subjective insights.

```
Do you think microfrontends are worth it?
```

4) Recommended PRO Structure

```
Role:
Act as a senior frontend engineer.

Objective:
Analyze and optimize the performance of this React component.

Context:
- App built with Vite + TypeScript
- Large list with frequent re-renders
- No external libraries allowed

Input:
### Component ###
[code here]
###

Constraints:
- Do not use external libraries
- Explain trade-offs
- State when NOT to use the solution

Output format:
1. Diagnosis
2. Possible causes
3. Proposed solution
4. Improved code
5. Trade-offs
6. Final validation

Validation:
If information is missing, assume the minimum and continue.
Clearly state your assumptions.
```

5) Roles in Chat Prompts

System

Defines general behavior, boundaries, tone, and output format.

```
You are a technical assistant.
If there is not enough information, ask or say "I don't know".
```

User

Contains the request, context, and materials.

```
Analyze this Vite compilation error...
```

Assistant

Can be used for previous examples or conversation history.

```
Example response showing the desired format.
```

> A system message helps steer the model, but it does not guarantee full compliance. That is why testing, measuring, iterating, and adding validation are still important.

6) Design Patterns That Work

- **Put the instruction first** whenever possible.
- **Use delimiters** to separate context, data, and rules: `###`, `<data>`, etc.
- **Specify the output contract**: JSON, bullets, table, steps, checklist.
- **Define fallback behavior**: what to do if information is missing or the request is out of scope.
- **Break down complex tasks** into subtasks if one prompt becomes too large.
- **Iterate**: test a simple version → improve context → add examples → refine format.

```
### Instruction ###
Extract the entities from the text.

### Format ###
Return ONLY JSON with this structure:
{
  "people": [],
  "companies": [],
  "places": []
}

### Text ###
[text here]
###
```

7) Prompt Types by Goal

❓ Direct Question

To get a concrete answer.

```
What is the difference between SQL and NoSQL?
```

📋 Instruction

To ask for a specific task.

```
Explain React hooks with simple examples.
```

🧲 Extraction

To convert free text into structure.

```
Extract customer_name, invoice_date, and total_amount as JSON.
```

🔍 Classification

To assign predefined labels.

```
Classify the sentiment:
Positive | Neutral | Negative
```

✍️ Transformation

To summarize, translate, rewrite, or change tone.

```
Rewrite this email with a professional and firm tone.
```

💡 Ideation

For brainstorming and exploration.

```
Suggest 10 project ideas using React + TypeScript.
```

8) Zero-shot, Few-shot, and Chain-of-Thought

🔹 Zero-shot

Give only the instruction, with no examples. Works well for simple or common tasks.

```
Classify the sentiment as Positive, Neutral, or Negative.
Text: "The product is okay, but I expected more."
```

🔹 Few-shot

Add examples to control format or improve consistency in more sensitive tasks.

```
Input: "Hello, send it today"
Output: "Could you please send it today?"

Input: "Send me the file now"
Output:
```

🔹 Chain-of-Thought (CoT)

Useful for step-by-step reasoning, debugging, or problems with multiple stages.

```
Solve the problem step by step and then provide the final answer.
```

> If zero-shot does not work well, try few-shot. If few-shot still fails on complex tasks, break the task down or use chaining.

> Important note: in some modern reasoning models, overloading the prompt with complex techniques may not help. Clear and simple instructions often work best.

9) Prompt Chaining

When a task is too large for a single prompt, it is often better to divide it into steps.

Step 1

Extract relevant information.

```
Extract the relevant quotes from the document to answer the question.
```

Step 2

Use the extracted content to build the response.

```
Based on the extracted quotes, answer the question with a clear and precise tone.
```

> Benefits: more control, more transparency, easier debugging, and better reliability.

10) Generated Knowledge and RAG

🧠 Generated Knowledge

First, ask the model to generate or list relevant knowledge, then reuse that information in a second prompt.

```
Step 1:
Generate 5 key facts about Kubernetes for a frontend developer.

Step 2:
Use ONLY those facts to write a short and clear explanation.
```

📚 RAG (Retrieval-Augmented Generation)

Instead of relying only on the model's internal knowledge, retrieve information from documents or external sources and ask it to answer based on them.

```
Answer ONLY with the information included between ###.
If the answer is not found, say "Not found in the context".

###
[document or excerpts]
###
```

> Use RAG when you need fresh data, internal documents, policies, or verifiable content.

11) Cues, Delimiters, and Output Contracts

🔹 Cues

A cue is a hint that helps the model start the output with the right structure.

```
Summarize the email.
Key points:
1.
```

🔹 Delimiters

```
### Instruction ###
Summarize the following text in 3 bullet points.

### Text ###
[content]
###
```

🔹 Output Contract

```
Return ONLY this JSON:
{
  "summary": "",
  "risks": [],
  "next_steps": []
}
```

12) Best Practices

- Start simple and improve iteratively.
- Be specific, but avoid irrelevant detail.
- Use examples when format matters.
- Define tone, audience, and technical level if that affects the answer.
- Specify what to do if context is missing.
- Avoid contradictory instructions.
- Validate critical outputs: AI can sound confident and still be wrong.

> Practical rules: "be clear", "define the format", "delimit the context", "iterate", "validate".

13) Common Mistakes

❌ Vague Prompt

```
Explain hooks
```

✅ Better Version

```
Explain useEffect, useMemo, and useCallback for a junior developer.
Include examples and when NOT to use each one.
```

❌ No expected format

```
Analyze this text
```

✅ Better Version

```
Analyze this text and return:
1. Summary
2. Risks
3. Suggested actions
```

> Avoid mixing "be brief" with "be comprehensive" unless you clearly prioritize one.

14) Useful Templates for Development

🐛 Debugging

```
Act as a senior engineer.

Analyze this error:
### ERROR ###
[paste the error]
###

Context:
- React + Vite + TypeScript

Return:
1. Possible causes
2. How to reproduce
3. Recommended solution
4. Trade-offs
5. What to verify afterward
```

🔍 Code Review

```
Review this code.

Evaluate:
- Readability
- Performance
- Potential bugs
- Maintainability
- Basic security

Deliver:
- Findings
- Severity
- Recommendation
- Suggested code
```

⚡ Code Generation

```
Create a custom React + TypeScript hook for data fetching.

Requirements:
- loading
- error
- cancellation with AbortController
- no any types

Return:
- code
- explanation
- trade-offs
```

🏗️ Architecture

```
Design a strategy for managing global state in a medium-sized React app.

Compare:
- Context API
- Zustand
- Redux Toolkit

Return:
- when to use each option
- pros and cons
- final recommendation
```

15) Document QA and Controlled Extraction

📄 Question Answering with Context

```
Answer ONLY using the content between ###.
If the answer does not appear, respond: "It is not found in the document".

###
[text or document]
###

Question:
[your question]
```

🧲 Structured Extraction

```
Extract the following information from the text:

- customer_name
- invoice_date
- total_amount

Return ONLY JSON with exactly these keys:
{
  "customer_name": null,
  "invoice_date": null,
  "total_amount": null
}
```

16) Final Quality Checklist

- ☑ Is the task clear?
- ☑ Is there enough relevant context?
- ☑ Is the input clearly delimited?
- ☑ Is the output format specified?
- ☑ Are useful constraints included?
- ☑ Is it clear what to do if information is missing?
- ☑ Does the prompt avoid contradictions?
- ☑ Does the output require human validation?

17) Master Reference Prompt

```
Act as a senior frontend engineer.

Objective:
Optimize the performance of a React component.

Context:
- React + Vite + TypeScript
- Large list
- Frequent re-renders
- No external libraries

Input:
### Component ###
[code here]
###

Constraints:
- Do not use external dependencies
- Explain trade-offs
- State when NOT to apply the solution
- If data is missing, assume the minimum and continue

Output format:
1. Diagnosis
2. Possible causes
3. Solution
4. Proposed code
5. Trade-offs
6. What to validate afterward

Validation:
Review your own solution and correct weaknesses if you detect them.
```

Built for studying and applying Prompt Engineering in real-world work, learning, and development scenarios.

Author **MC. David Alejandro Garcia Garcia**